UNIT-5.1 Delegates

Introduction To Delegates-

A delegate is an object which refers to a method

OR

Delegate is a reference type variable that can hold a reference to the methods.

Delegates in C# are similar to the **function pointers in C/C++.**

It provides a way which tells which method is to be called when an event is triggered.

Delegate Declaration-

Delegate type can be declared using the delegate keyword. Once a delegate is declared, delegate instance will refer and call those methods whose return type and parameter-list matches with the delegate declaration.

Syntax:

[access_modifier] delegate [return_type] [delegate_name] ([parameter_list]);

Example:

public delegate int del_add (int a, int b, int c);

Note: A delegate will call only a method which agrees with its signature and return type. A method can be a static method associated with a class or can be an instance method associated with an object, it doesn't matter.

Instantiation & Invocation of Delegates

After declaring a delegate, a delegate object is created with the help of ***new*** keyword. Once a delegate is instantiated, a method call made to the delegate is pass by the delegate to that method. The parameters passed to the delegate by the caller are passed to the method, and the return value, if any, from the method, is returned to the caller by the delegate.

This is known as invoking the delegate.

Syntax:

[delegate_name] [instance_name] = new [delegate_name](calling_method_name);

Program For Delegate Implementation-

```
using System;
namespace DelegateDemo
{
 public delegate void MyDelegate(int x, int y );//Delegate
Declaration

 class Program
    {
        public static void add(int x,int y)  //Static Method
        {
            Console.WriteLine("Addition=" + (x + y));
        }
        public void sub(int x,int y)  //Non Static Method
        {
            Console.WriteLine("Subtraction=" + (x - y));
        }

 static void Main(string[] args)
        {
 Program pob = new Program();//Class Instanciation
 MyDelegate md = new MyDelegate(Program.add);//Delegate
Instanciation

md(10, 5); //Delegate Invokation

 MyDelegate nd = new MyDelegate(pob.sub);//Delegate
Instanciation

nd.Invoke(15, 10);//Delegate Invokation

 Console.ReadKey();
        }
    }
}
```

Output-

Addition=15
Subtraction=5

Types Of Delegates-

1.Singlecast Delegate-

A delegate that points Single method is called a Singlecast delegate.

Above Program is an example of Singlecast delegate.

2.Multicast Delegate

A delegate that points multiple methods is called a Multicast delegate.

The "+" or "+=" operator *adds a function to the invocation list*, and the "-" and "-=" operator *removes it.*

Multicast Delegates also called as Combinable Delegates.

While using Multicast Delegates we must satisfy Following Conditions,

1. *The return type of the Delegate must be void.*
2. *None of the parameters of the Delegate type can be declared as the output parameters, using out keywords.*

Example

```
using System;
namespace MultiCastDeligateDemo
{
public delegate void MathDel(int a, int b);
class Program
    {
public void sum(int x,int y)
{
     Console.WriteLine("{0} + {1} = {2}", x, y, x + y);
}
public void sub(int x, int y)
{
     Console.WriteLine("{0} - {1} = {2}", x, y, x - y);
}
public void mul(int x, int y)
{
     Console.WriteLine("{0} * {1} = {2}", x, y, x * y);
}
```

```
public void div(int x, int y)
{
     Console.WriteLine("{0} / {1} = {2}", x, y, x / y);
}

static void Main(string[] args)
        {
            Program pob = new Program();
            MathDel ad = new MathDel(pob.sum);
            MathDel sb = new MathDel(pob.sub);
            MathDel ml = new MathDel(pob.mul);
            MathDel dv = new MathDel(pob.div);

            MathDel multi = ad + sb + ml + dv;
            multi(20, 10);

            multi -= sb;
            multi.Invoke(50, 25);
            Console.ReadKey();

        }
    }
}
```

Output-

```
20 + 10 = 30
20 - 10 = 10
20 * 10 = 200
20 / 10 = 2

50 + 25 = 75
50 * 25 = 1250
50 / 25 = 2
```

Anonymous Method-

An anonymous method is an inline method without a name.

It is created using the *delegate* keyword and doesn't require a name and return type.

Hence we can say, an anonymous method has the only body without a name, optional parameters and return type.

An anonymous method behaves like a regular method and allows us to write inline code in place of explicitly named methods.

Syntax: **delegate (parameter list)**

{
// Code..
};

Example-

```
namespace AnanymousMethodDemo
{
public delegate void MyDel(int a, int b);//Delegate
Declaration

class Program
 {
  static void Main(string[] args)
     {
MyDel md = delegate (int a, int b) //Ananymous Method Def.
      {
       Console.WriteLine("ADD=" + (a + b));
      };

md(20, 10);

Console.ReadKey();
         }
    }
}
```

Output-

ADD=30